

UNIVERSIDAD AUTÓNOMA DE MADRID

ESCUELA POLITÉCNICA SUPERIOR



Práctica Probabilidad y Estadística

Probabilidad y Estadística 2022/2023

Alejandro Díez, Mario García y Javier Pérez

Grupo 226

Equipo 3

Índice:

Práctica 1:	2
Ejercicio 1:	2
Parte 1: Importación de los datos a RStudio. Limpieza de los datos.	2
Parte 2: Análisis Descriptivo:	3
Ejercicio 2:	7
Parte 3: Análisis Bivariante	7
Práctica 2:	9
Ejercicio 1:	9
Ejercicio 2:	12

Práctica 1:

Ejercicio 1:

Parte 1: Importación de los datos a Rstudio. Limpieza de los datos.

Para importar los datos de googleplay.csv tenemos en cuenta que:

- los datos están separados por “;”
- Los decimales están separados por “.”
- Los valores “missings” o valores perdidos están denotados en la base de datos por “NaN”.

Esto se consigue con la siguiente línea:

```
read.csv("googleplay.csv", sep=";", dec=".", na.strings="NaN", stringsAsFactors = FALSE)
```

En ella indicamos la primera condición mediante “;” debido a que los datos están separados por “;”, la segunda condición con “.” debido a que los decimales están separados de las unidades por un punto, la tercera con “NaN” debido a que así se denotan los datos sin valores y por último se usa el `stringsAsFactors = FALSE` para que no lea las variables string como factores. Cada uno de estos parámetros que usamos en la función `read` sirve para indicar propiedades especiales del texto.

Posteriormente indicamos cuáles son variables numéricas mediante las siguientes líneas:

```
googlPl$Rating <- as.numeric(googlPl$Rating)
googlPl$Reviews <- as.numeric(googlPl$Reviews)
googlPl$Price <- as.numeric(googlPl$Price)
```

Por último, eliminamos todas las filas con valores faltantes:

```
googlPl = na.omit(googlPl)
```

Para comprobar que la tabla es correcta imprimimos las dimensiones de la tabla y lo comparamos con las dimensiones del enunciado (8724 filas y 6 columnas):

```
var_num <- sapply(googlPl, is.numeric)
cat("Variables numéricas:", names(googlPl[var_num]), "\n")
```

Parte 2: Análisis Descriptivo:

1. Para obtener la tabla de frecuencias de la variable Category utilizamos:

```
table(googIPI$Category)
```

Esto filtra las columnas de nuestros datos para generar una tabla con las frecuencias de la variable deseada.

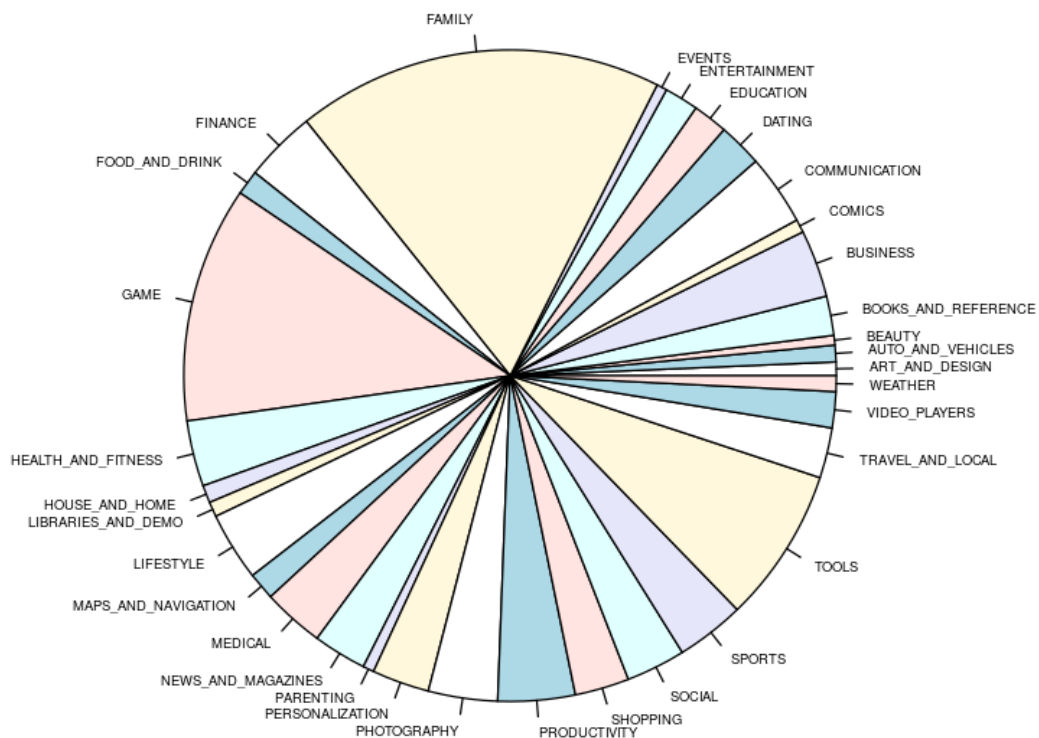
Con la tabla preparada, usamos la siguiente función para generar el diagrama de sectores:

```
pie(table(googIPI$Category), cex = 0.47, main = "Categorías de aplicaciones en Google Play Store")
```

Usamos "cex" para adaptar el tamaño de letra a 0.47 puntos, a fin de mejorar el aspecto del gráfico. Además le ponemos un título con "main".

Obtenemos el siguiente resultado:

Categorías de aplicaciones en Google Play Store



2. Para obtener la categoría con más aplicaciones usamos:

```
max <- sort(table(googIPl$Category), decreasing = TRUE)[1]  
cat("La categoría con más aplicaciones es:", names(max), "\n")
```

Para obtener las 5 categorías con menos aplicaciones usamos:

```
min <- sort(table(googIPl$Category), decreasing = FALSE)[1:5]  
cat("Las 5 categorías con menos aplicaciones son:", names(min), "\n")
```

Para obtener las 5 categorías con más aplicaciones usamos:

```
max <- sort(table(googIPl$Category), decreasing = TRUE)[1:5]  
cat("Las 5 categorías con más aplicaciones son:", names(max), "\n")
```

El resultado es el siguiente:

```
La categoría con más aplicaciones es: FAMILY  
Las 5 categorías con menos aplicaciones son: BEAUTY EVENTS PARENTING COMICS ART AND DESIGN  
Las 5 categorías con mayor disponibilidad de aplicaciones son: FAMILY GAME TOOLS PRODUCTIVITY FINANCE
```

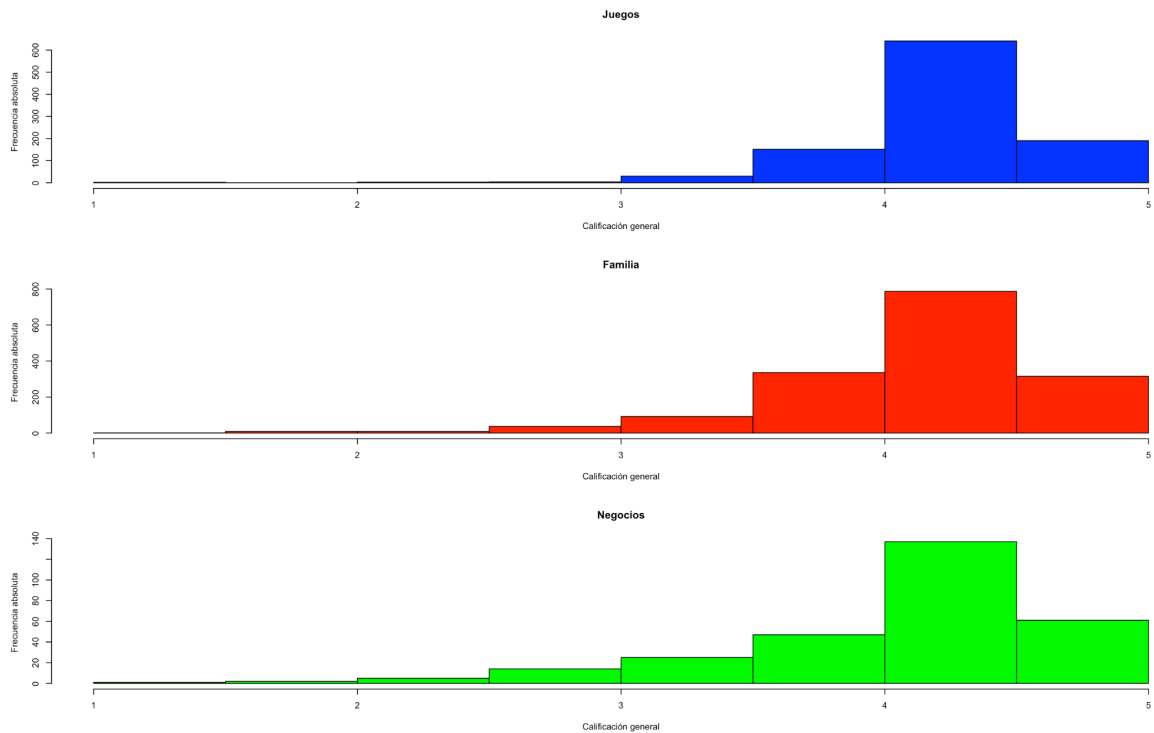
3. Elegimos las categorías “Game”, “Family” y “Bussines” mediante estas líneas:

```
game <- subset(googIPl, Category == "GAME")  
family <- subset(googIPl, Category == "FAMILY")  
business <- subset(googIPl, Category == "BUSINESS")
```

A continuación creamos sus histogramas (y los coloreamos) usando

```
par(mfrow = c(3, 1))  
hist(game$Rating, main = "Juegos", xlab = "Calificación general", ylab = "Frecuencia  
absoluta", col="blue")  
hist(family$Rating, main = "Familia", xlab = "Calificación general", ylab = "Frecuencia  
absoluta", col="red")  
hist(business$Rating, main = "Negocios", xlab = "Calificación general", ylab = "Frecuencia  
absoluta", col = "green")
```

Obteniendo



4. Para obtener el nuevo subconjunto con todos los datos, usamos subset de esta manera:

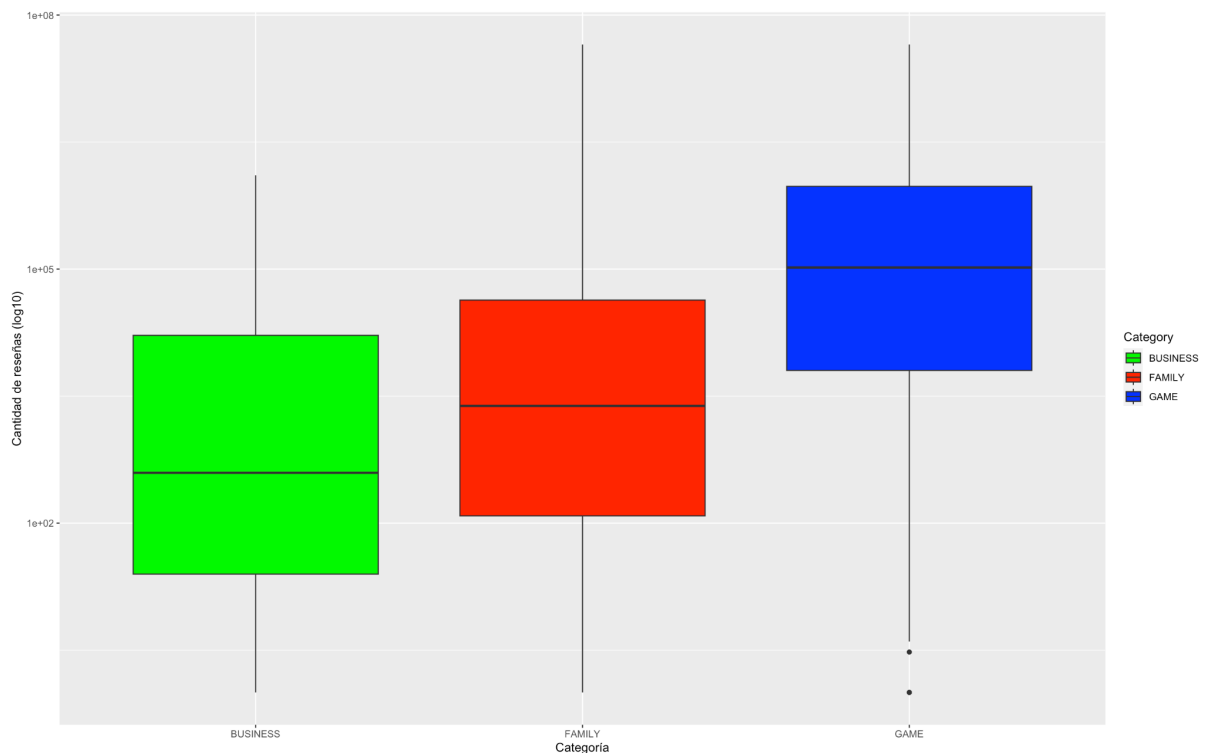
```
subset_googlPl <- subset(googlPl, Category %in% c("GAME", "FAMILY", "BUSINESS"))
```

Con el uso de %in% obtenemos todos los valores, filtrados por las categorías elegidas en el ejemplo anterior.

5. Para generar la gráfica, importamos la librería “ggplot2” y creamos este gráfico conjunto mediante:

```
ggplot(subset_googlePl, aes(x = Category, y = Reviews)) +  
  geom_boxplot(aes(fill = Category)) +  
  scale_y_log10() +  
  scale_fill_manual(values = c("GAME" = "blue", "FAMILY" = "red", "BUSINESS" =  
    "green")) +  
  labs(x = "Categoría", y = "Cantidad de reseñas (log10)")
```

Generándose esto:



Se observa que las tres categorías tienen una gran cantidad de datos atípicos, especialmente la categoría "GAME". Además, en todas las categorías se observa una gran asimetría hacia valores altos de cantidad de reseñas, lo que sugiere que la mayoría de las aplicaciones tienen pocas reseñas y solo un pequeño número de aplicaciones tienen muchas reseñas.

Ejercicio 2:

Parte 3: Análisis Bivariante

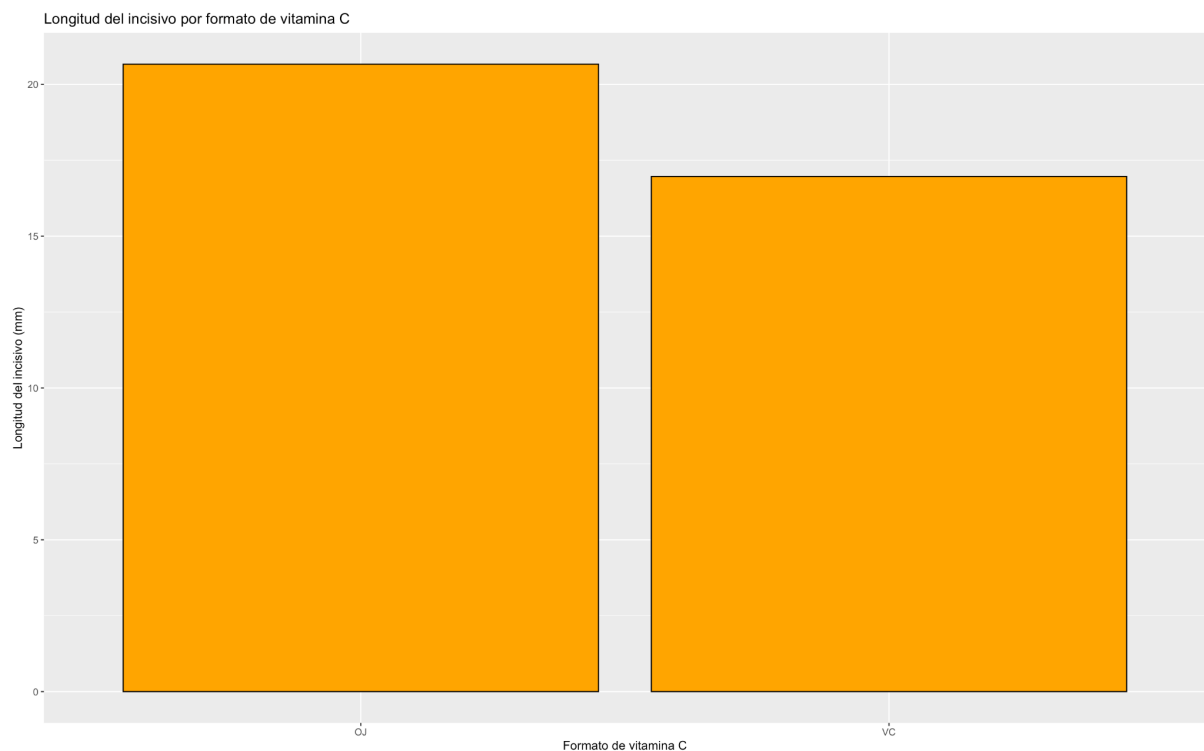
1. Para obtener los datos de “vitamin.dat” utilizamos read.table de esta manera:

```
datos_vitaminas<- read.table("vitamin.dat", header=TRUE)
```

Mientras que para generar el gráfico, vamos a volver a usar “ggplot2” así:

```
ggplot(datos_vitaminas, aes(x=supp, y=len)) +  
  geom_bar(stat="summary", fun="mean", fill="orange", color="black") +  
  ggtitle("Longitud del incisivo por formato de vitamina C") +  
  xlab("Formato de vitamina C") +  
  ylab("Longitud del incisivo (mm)")
```

Generando a su vez este gráfico:



En el cual podemos observar una clara diferencia entre la longitud de los incisivos, siendo mayor en los niños suministrados con vitamina en formato Zumo de Naranja.

2. Las correlaciones lineales las obtenemos mediante la función “cor” y os valores a correlar. Para el primero, la longitud de los incisivos en relación con la dosis de ácido ascórbico es de 0.8923367, que obtenemos mediante:

```
cor_ascorbico <- cor(datos_vitaminas$len[datos_vitaminas$supp == "VC"],  
  datos_vitaminas$dose[datos_vitaminas$supp == "VC"])
```

Mientras que para los incisivos en relación con la dosis de Zumo de Naranja suministrado será de 0.7889725, obtenido a su vez mediante:

```
cor_zumo <- cor(datos_vitaminas$len[datos_vitaminas$supp == "OJ"],  
  datos_vitaminas$dose[datos_vitaminas$supp == "OJ"])
```

3. En base a los resultados anteriores, presenta más correlación la dosis de vitamina C administrada en formato ácido ascórbico que la dosis administrada en formato Zumo de Naranja, indicando que la primera tiene una relación lineal más fuerte.

4. Para ajustar la recta de regresión lineal usamos la función “lm”, una función general usada para ajustar modelos lineales, de esta manera:

```
recta_regresion <- lm(len ~ dose, data = datos_vitaminas)
```

Intercepto: 5.49122 y Pendiente: 9.807691, además con un coeficiente de determinación (R^2): 0.6889495, por lo tanto, ya que no es superior a 0.8, no es un buen ajuste.

Práctica 2:

Ejercicio 1:

Como siempre, importamos los datos de “SOCR.dat” con “read.table”

1. Si queremos poner dos gráficos juntos, debemos usar la función “par”, modificando el mfrow para que ponga 1 fila con 2 gráficas:

```
par(mfrow = c(1, 2))
```

Ahora para dibujar cada histograma y su línea de densidad normal asociada hacemos esto:

- Historiograma del peso

```
hist(data_SOCR$weight, main = "historiograma peso", xlab = "Libras", ylab = "Densidad",  
      freq = FALSE)
```

- Curva de densidad de la normal asociada al historiograma peso

```
x_weight = seq(min(data_SOCR$weight), max(data_SOCR$weight), length = 100)  
y_weight = dnorm(x_weight, mean(data_SOCR$weight), sd(data_SOCR$weight))  
lines(x_weight, y_weight, col = "red")
```

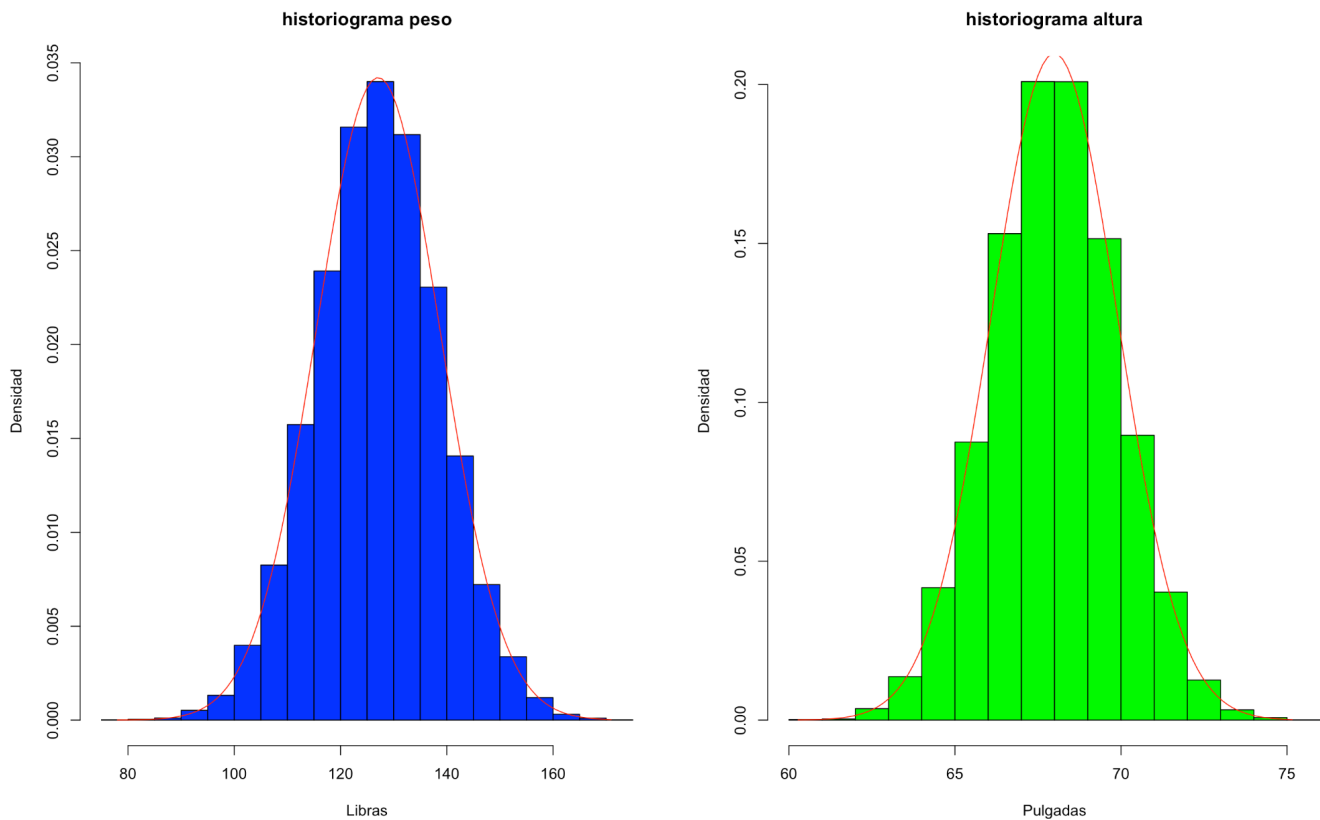
- Historiograma de altura

```
hist(data_SOCR$height, main = "historiograma altura", xlab = "Pulgadas", ylab =  
      "Densidad", freq = FALSE)
```

- Curva de densidad de la normal asociada al historiograma altura

```
x_height = seq(min(data_SOCR$height), max(data_SOCR$height), length = 100)  
y_height = dnorm(x_height, mean(data_SOCR$height), sd(data_SOCR$height))  
lines(x_height, y_height, col = "red")
```

Obteniendo este resultado:



2. Para las medias usaremos “mean” y para la desviación estándar usaremos “sd” de esta manera:

- Media y desviación estándar de la altura:

```
mean(data_SOCR$height)
```

```
sd(data_SOCR$height)
```

Cuyos resultados son 67.99311 y 1.901679 respectivamente.

- Media y desviación estándar del peso

```
mean(data_SOCR$weight)
```

```
sd(data_SOCR$weight)
```

Cuyos resultados son 127.0794 y 11.6609 respectivamente.

3. Para calcular la probabilidad de que una persona tenga una altura entre 66 y 69 pulgadas.

```
pnorm(69, mean(data_SOCR$height), sd(data_SOCR$height)) - pnorm(66,  
  mean(data_SOCR$height), sd(data_SOCR$height))
```

Para calcular la probabilidad de que una persona tenga un peso mayor que 134 libras.

```
1 - pnorm(134, mean(data_SOCR$weight), sd(data_SOCR$weight))
```

Considerando un grupo de 20 personas seleccionadas al azar del municipio de Vizcaya. Calculamos la probabilidad de que al menos 4 tengan una altura de más de 50 pulgadas con:

```
Nrow = nrow(data_SOCR)  
1 - pbinom(3, 20, sum(data_SOCR$height > 50)/Nrow)
```

Considerando el mismo grupo de 20 personas. Calculamos cual es la probabilidad de que al menos 4 tengan una altura mayor que 70 pulgadas y que al menos 11 midan menos de 70 pulgadas con:

```
sumMas70 = sum(data_SOCR$height > 70)  
p1 = sum(dhyper(4:20, sumMas70, Nrow-sumMas70, 20))  
p2 = sum(dhyper(0:9, Nrow-sumMas70, sumMas70, 20))  
p3 = sum(dhyper(0:10, Nrow-sumMas70, sumMas70, 20))  
p4 = sum(dhyper(0:11, Nrow-sumMas70, sumMas70, 20))  
ptotal = p1 + p2 + p3 + p4
```

Calculamos el cuantil 0.10 de la variable peso y el cuantil 0.60 de la variable altura con:

```
quantile(data_SOCR$weight, 0.1)  
quantile(data_SOCR$height, 0.6)
```

Obtenemos los siguientes resultados:

```
Media Altura:[1] 67.99311  
Desviacion estandar altura:[1] 1.901679  
Media Peso:[1] 127.0794  
Desviacion estandar peso:[1] 11.6609  
Probabilidad altura entre 66 y 69:[1] 0.5544605  
Probabilidad peso mayor a 134:[1] 0.276428  
Probabilidad al menos 4 tengan altura mayor a 50:[1] 1  
Probabilidad al menos 4 tengan altura mayor a 70 y 11 menos de 70:[1] 0.3373269  
Cuantil 0.10 variable peso:      10%  
112.1098  
Cuantil 0.60 variable altura:      60%  
68.48009
```

Ejercicio 2:

1. Generamos una muestra aleatoria de tamaño 100000 a partir de la distribución Binomial(1000, 0.002) usando las siguientes líneas:

```
n = 100000
size = 10000
prob = 0.002
λ = 2
muestra_aleatoria = rbinom(n, size, prob)
```

Posteriormente dibujamos el histograma con:

```
hist(muestra_aleatoria, col = "blue", main = "Muestra Binomial(1000, 0.002)")
```

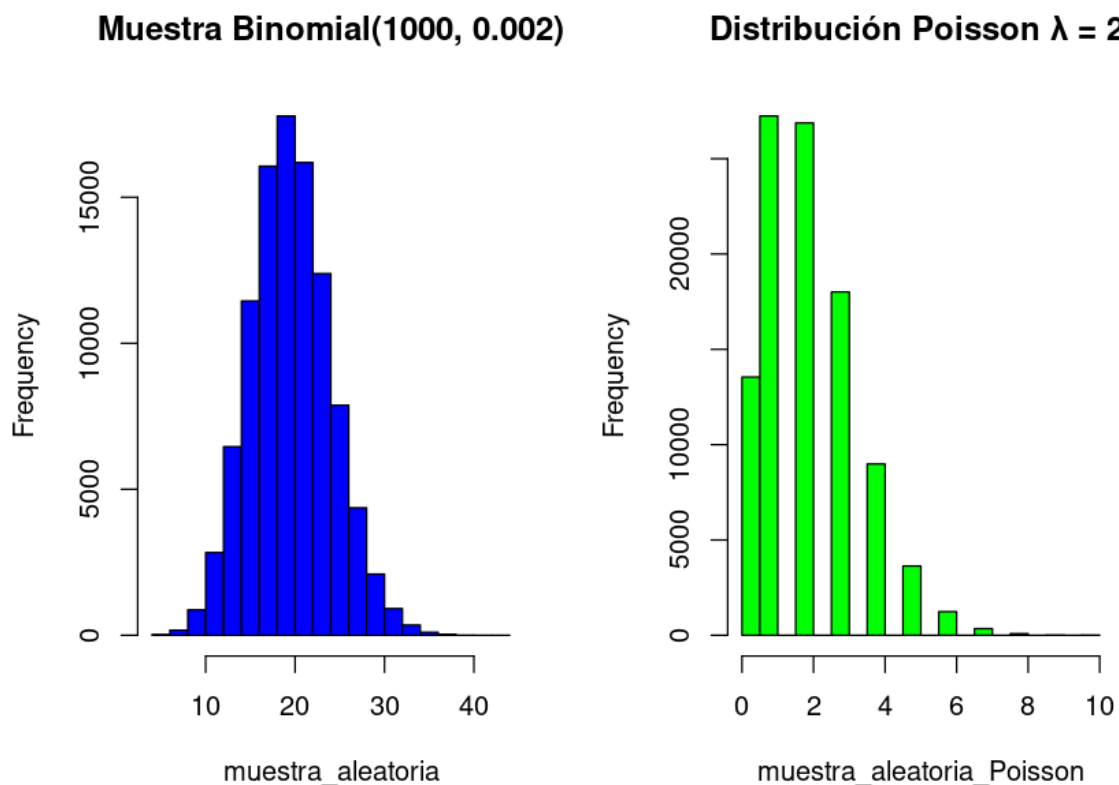
2. Ahora generamos una muestra aleatoria de tamaño 100000 a partir de la distribución Poisson con $\lambda = 2$:

```
muestra_aleatoria_Poisson = rpois(n, λ)
```

Y dibujamos el histograma correspondiente con:

```
hist(muestra_aleatoria_Poisson, col = "green", main = "Distribución Poisson  $\lambda = 2$ ")
```

El resultado de ambas gráficas es el siguiente:



3. Como se puede observar, la distribución binomial tiene una forma más simétrica, mientras que la Poisson tiene una forma más asimétrica y sesgada hacia la izquierda. Esto se debe a que la distribución Poisson se usa para otro tipo de experimentos, en los que los eventos ocurren de manera aleatoria en un intervalo continuo de tiempo o espacio. En este caso se debería utilizar la distribución binomial ya que el experimento está formado por ensayos independientes entre sí, que no son continuos en el tiempo. Teniendo un número de ensayos fijo tendremos que buscar el número de éxitos para ese número determinado de ensayos.